

HOME-WORK 4

Sunday, April 26, 2026 3:56 PM

Syed Swaleh Abbas, Syed Muhammad Haider Zaidi

Question 1 [20 Marks]

A 4 KiB cache can hold 4 KiB of data; however, caches are also required to store metadata such as tags and valid bits. We will see how block size affects the size of metadata needed for a direct-mapped cache. For all parts, assume that addresses are 64 bits, and we are dealing with words of 4 bytes.

a) Calculate the total number of bits required to implement a 128 KiB cache with 4 words per block. [5 Marks]

- block size = 4 words $\times$ 4 bytes/word = 16 bytes
- Number of blocks = $128 \times 1024 = 131,072$ bytes
which is,
 $131072/16 = 8192$ blocks
- Index bits = $\log_2(8192) = 13$ bits
- offset bits = $\log_2(16) = 4$ bits
- Tag bits = $64 - 13 - 4 = 47$ bits
- Line size = Data (4 words $\times$ 32 bits = 128 bits) + Tag (47 bits) + Valid bit (1 bit) = 176 bits
- Total cache size: $8192 \text{ blocks} \times 176 \text{ bits/block} = 1441792 \text{ bits}$

b) Now repeat the calculation in part a with 8, 16, 32 and 64 words per block. [10 Marks]

Block size	8 words/block	16 words/ block	32 words/block	64 words/block
# of cache blocks	4096	2048	1024	512
Bits for Index	12	11	10	9
Bits for Tag	47	47	47	47
Line Size in bits	304	560	1072	2096
Total Size in bits	1245184	1146880	1097728	1073152

c) Find the ratio of total cache size to the amount of data (i.e. data size) for all the cases in part 'a' and 'b'. [5 Marks]

- We will divide Total size of every part with: 1048576 bits,

Block Size	Ratio
4 words/blocks	1.375
8 words/block	1.187
16 words/block	1.094
32 words/block	1.047
64 words/block	1.023

$$\therefore 128 \text{ kib} \times 1024 \text{ bytes/kib} \times 8 \text{ bits/byte} = 1048576 \text{ bits}$$

Question 3 [10 Marks]

Cache block size (B) can affect both miss rate and miss latency. Assuming a machine with a base CPI of 1, and an average of 1.35 references (both instruction and data) per instruction, find the block size that minimizes the total miss latency given the following miss rates for various block sizes. Cache block size is assumed to be in bytes.

8: 10%	16: 8%	32: 7%	64: 5%	128: 3%
--------	--------	--------	--------	---------

a) What is the optimal block size for a miss latency of $20 \times B$ cycles? [5 Marks]

- Miss latency = $20 \times B$ cycles:

$$B=8: \text{latency} = 160, \text{penalty} = 0.1 \times 160 = 16.0$$

$$B=16: \text{latency} = 320, \text{penalty} = 0.08 \times 320 = 25.6$$

$$B=32: \text{latency} = 640, \text{penalty} = 0.07 \times 640 = 44.8$$

$$B=64: \text{latency} = 1280, \text{penalty} = 0.05 \times 1280 = 64.0$$

$$B=128: \text{latency} = 2560, \text{penalty} = 0.03 \times 2560 = 76.8$$

- as block size 8 minimizes miss penalty so it's the optimal size.

b) What is the optimal block size for a miss latency of $24 + B$ cycles? [5 Marks]

$$\begin{aligned}
 B=8 : \text{Latency} &= 32, \text{Penalty} = 0.1 \times 32 = 3.2 \\
 B=16 : \text{Latency} &= 40, \text{Penalty} = 0.08 \times 40 = 3.2 \\
 B=32 : \text{Latency} &= 56, \text{Penalty} = 0.07 \times 56 = 3.92 \\
 B=64 : \text{Latency} &= 88, \text{Penalty} = 0.05 \times 88 = 4.4 \\
 B=128 : \text{Latency} &= 152, \text{Penalty} = 0.03 \times 152 = 4.56
 \end{aligned}$$

- Block size of 8 bytes or 16 bytes are optimal sizes as they yield minimum Miss penalty.

Question 4 [10 Marks]

We are given 4 arrays of size 6. Each element of an array is of 32 bytes i.e., one word. Following is the data stored in the array:

A = (27, 11, 23, 10, 28, 12)

B = (20, 35, 33, 48, 10, 32)

C = (40, 11, 9, 42, 49, 20)

D = (11, 31, 17, 37, 13, 2)

Data to be Inserted	Hit/Miss	Cache Index							
		0	1	2	3	4	5	6	7
A[0]	M	A[0]							
A[1]	M	A[0]	A[1]						
A[2]	M	A[0]	A[1]	A[2]					
A[1]	H	A[0]	A[1]	A[2]					
A[5]	M	A[0]	A[1]	A[2]			A[5]		
B[5]	M	A[0]	A[1]	A[2]			B[5]		
B[4]	M	A[0]	A[1]	A[2]		B[4]	B[5]		
B[3]	M	A[0]	A[1]	A[2]	B[3]	B[4]	B[5]		
B[3]	H	A[0]	A[1]	A[2]	B[3]	B[4]	B[5]		
B[4]	H	A[0]	A[1]	A[2]	B[3]	B[4]	B[5]		
D[1]	M	A[0]	D[1]	A[2]	B[3]	B[4]	B[5]		
D[2]	M	A[0]	D[1]	D[2]	B[3]	B[4]	B[5]		
D[3]	M	A[0]	D[1]	D[2]	D[3]	B[4]	B[5]		
D[4]	M	A[0]	D[1]	D[2]	D[3]	D[4]	B[5]		
C[3]	M	A[0]	D[1]	D[2]	C[3]	D[4]	B[5]		
C[2]	M	A[0]	D[1]	C[2]	C[3]	D[4]	B[5]		
C[4]	M	A[0]	D[1]	C[2]	C[3]	C[4]	B[5]		
C[2]	H	A[0]	D[1]	C[2]	C[3]	C[4]	B[5]		

- Hit ratio = 4 hits / 18 accesses = 22.2%
- Miss ratio = 14 misses / 18 accesses = 77.8%

Question 5 [10 Marks]

Repeat the same task as in Question 4 but now the cache has 4 lines and the block size has increased to two words. The first insertion is done again so that you may get the idea.

Data to be Inserted	Hit/Miss	Cache Index							
		0		1		2		3	
A[0]	M	A[0]	A[1]						
A[1]	H	A[0]	A[1]						
A[2]	M	A[0]	A[1]	A[2]	A[3]				
A[1]	H	A[0]	A[1]	A[2]	A[3]				
A[5]	M	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]		
B[5]	M	A[0]	A[1]	A[2]	A[3]	B[4]	B[5]		
B[4]	H	A[0]	A[1]	A[2]	A[3]	B[4]	B[5]		
B[3]	M	A[0]	A[1]	B[2]	B[3]	B[4]	B[5]		
B[3]	H	A[0]	A[1]	B[2]	B[3]	B[4]	B[5]		
B[4]	H	A[0]	A[1]	B[2]	B[3]	B[4]	B[5]		
D[1]	M	D[0]	D[1]	B[2]	B[3]	B[4]	B[5]		
D[2]	M	D[0]	D[1]	D[2]	D[3]	B[4]	B[5]		
D[3]	H	D[0]	D[1]	D[2]	D[3]	B[4]	B[5]		
D[4]	M	D[0]	D[1]	D[2]	D[3]	D[4]	D[5]		
C[3]	M	D[0]	D[1]	C[2]	C[3]	D[4]	D[5]		
C[2]	H	D[0]	D[1]	C[2]	C[3]	D[4]	D[5]		
C[4]	M	D[0]	D[1]	C[2]	C[3]	C[4]	C[5]		
C[2]	H	D[0]	D[1]	C[2]	C[3]	C[4]	C[5]		

What is the Hit Ratio and the Miss Ratio in this case? Is it better than the previous? Does loading the whole array into the cache help us in accessing the elements fast?

- Hit ratio = $8/18 = 44.4\%$
- Miss ratio = $10/18 = 55.6\%$
- Yes, increasing the block size increased spatial locality, which dropped the miss rate from 77.8% to 55.6%.
- Loading a full array at a time would eliminate conflict misses that occur when elements of different arrays map to the same cache line.

Question 2 [10 Marks]

Considering the address size of 32-bits, fill in the data for different types of caches

	Blocks	Data per block	Sets	Associativity -ways	Tag Bits	Index Bits	Offset Bits
Fully Associative Cache	32	4 words	--	32	28	0	4
Direct Mapped Cache	32	8 words	--	1	22	5	5
Set Associative Cache	32	8 words	4	8	25	2	5
Direct Mapped Cache	64	4 words	--	1	22	6	4
Set Associative Cache	128	8 words	32	4	22	5	5
Set Associative Cache	256	8 words	32	8	22	5	5
Fully Associative Cache	512	8 words	--	512	27	0	5
Direct Mapped Cache	1024	8 words	--	1	17	10	5
Set Associative Cache	2048	4 words	64	32	22	6	4
Direct Mapped Cache	4096	8 words	--	1	15	12	5

Question 6 [20 Marks]

Assume that main memory accesses take 70 ns and that 36% of all instructions access data memory. The following table shows data for L1 caches attached to each of two processors, P1 and P2.

	L1 size	L1 miss rate	L1 hit time
P1	2 KiB	8%	.66 nsec
P2	4 KiB	6%	.9 nsec

- a) If the L1 hit time determines the cycle times for P1 and P2, what are their respective clock rates? [2 Marks]

QUESTION #6

(a)

For P1

$$\begin{aligned} \text{P1 Clock Rate} &= 1/0.66 \text{mSec} \\ &= 1.5151 \text{ GHz} \end{aligned}$$

For P2

$$\begin{aligned} \text{P2 Clock Rate} &= 1/0.9 \text{mSec} \\ &= 1.11 \text{ GHz} \end{aligned}$$

(b)

$$\therefore \text{AMAT} = 1(\text{Hit time}) + \text{Miss Rate} \times \text{Miss Penalty}$$

(Avg memory Access Time)

$$\begin{aligned} \text{P1: miss penalty} &= \frac{70 \text{ ns}}{0.66 \text{ ns/cycle}} \\ &= 106.0606 \text{ cycles} \end{aligned}$$

So, assuming a cache hit takes 1 cycle

$$\begin{aligned} \text{AMAT}_{P1} &= 1 + (0.08 \times 106.06) \\ &= 9.4848 \text{ cycles} \end{aligned}$$

And,

$$\begin{aligned} \text{AMAT}_{P2} &= 1 + (0.06 \times 77.78) \\ &= 5.67 \text{ cycles} \end{aligned}$$

(c)

$$\text{Total CPI} = \text{Base CPI} + \left(\frac{\text{memory accesses per instruction} \times \text{Average stall cycles}}{\text{Average stall cycles}} \right)$$

Average stall cycle is exactly $\text{AMAT} - 1$ (we're subtracting 1 because of the first cycle of access is a hit and is already accounted for in the Base CPI)

$$\begin{aligned} \text{P1 CPI} &= 1 + 1.36(9.48 - 1) \\ &= 12.5378 \text{ cycles/instructions} \end{aligned}$$

$$\begin{aligned} \text{P2 CPI} &= 1 + 1.36(5.67 - 1) \\ &= 7.3512 \text{ cycles/instruction} \end{aligned}$$

For which is faster

$$\text{Time} = \text{CPI} \times \text{Cycle Time}$$

$$\begin{aligned} \text{P1 Time} &= 12.54 \text{ cycles} \times 0.66 \text{ ns/cycle} \\ &= 8.2716 \text{ ns/instruction} \end{aligned}$$

$$\begin{aligned} \text{P2 Time} &= 7.3512 \times 0.9 \\ &= 6.61608 \text{ ns/instruction} \end{aligned}$$

Even though P1 has faster clock, P2 finishes instructions faster overall because P1 wastes too much time waiting on memory.

(d)

With addition of L2 cache,

$$AMAT = L1_{hit\ time} + L1_{miss\ rate} \times (L2_{hit\ time} + (L2_{miss\ rate} \times L2_{miss\ penalty}))$$

First we convert L2's hit time to P1's clock cycles
 $5.62 / 0.66 = 8.515$ cycles

So,

$$AMAT = 1 + 0.08[8.515 + (0.6)(106.06)] \\ = 6.772 \text{ cycles}$$

Thus, AMAT improved from 9.48 to 6.772.

(e)

$$CPI = 1 + 1.36(6.77 - 1) \\ = 8.85$$

Improved from 12.54 CPI to 8.85 CPI.

(f)

We know P1's original AMAT was 9.48 cycles
We want to find the L2 Miss Rate (M_{L2})

$$1 + 0.08[8.515 + (M_{L2} \times 106.06)] = 9.48$$

$$M_{L2} = 0.9191$$

The L2 cache ^{miss} rate needs to be worse than 91.91% for it to make system slower.

Page No. _____

VS

Signature _____

Question 7 [20 Marks]

A simple matrix transpose can be implemented as

for ($i = 0; i < m; i++$)

for ($j = 0; j < n; j++$)

Transpose[j][i] = Actual[i][j];

Where 'm' is the number of rows and 'n' is the number of columns in Actual.

Considering each matrix element is of 1 byte and is stored in row major order.

QUESTION #7

(a)

Spatial Locality: Present in Actual $[i][j]$.

The matrix is stored in row major order, and the inner loop iterates over columns (j)

Temporal Locality: Shown by loop control variables i and j , which are accessed repeatedly throughout the loop execution.

(b)

(i) In each iteration, the program reads one byte (Actual) and writes 1 byte (Transpose)

(ii) Total accesses = $2mn$

(c)

Without a cache, memory access latencies are uniform across all addresses. Therefore, changing the access pattern from row major to column major (or vice versa) would have no effect on the computation time of the code.

(d)

Yes, it will affect execution time. In the original loop, writer to Transpose jump across memory, causing a write miss on nearly every access (poor spatial locality). If the loops are interchanged, Transpose is written sequentially (good spatial locality), minimizing costly write misses, while ~~actual~~ Actual takes penalty of read misses. Since write misses are more expensive, this interchange will decrease execution time.

(e)

(*) The cache capacity is 16 KB

(*) The total working set involves both matrices, meaning $2m^2$ bytes of data.

(*) A capacity miss strictly occurs when the working set exceeds the cache size:

$$2m^2 > 16384$$

$$m^2 > 8192$$

$$m > 90.50966$$

(*) The minimum integer value for m is 91.

(f) The in-place transpose has a memory footprint of just m^2 bytes compared to $2m^2$ bytes for the original approach. Because the working set is cut exactly in half, the in-place method will suffer from fewer capacity misses for larger matrices.